

Tema ASC- Grupa 134

Stamin Daria-Alexandra

January 10, 2026

1 Introducere

În acest proiect am încercat să implementez codarea și decodarea pentru o variantă a cifurilor Playfair. Problema impune crearea unei matrici 5x5 formată din litere care nu se repetă. Vom introduce o cheie și apoi o umplem. Cu ajutorul acestei matrici putem encripta sau decripta un mesaj cu ajutorul a 3 reguli. Pentru a rezolva problema am împărțit-o în 4 secțiuni.

1. Construirea matricii
2. Procesarea mesajului
3. Codarea perechilor
4. Decodarea

2 Rezolvarea problemei

2.1 Construirea matricii

Problema impune o matrice de 5x5, inițial goală. O să o completăm de sus în jos cu litere din cheie, fără să se repete. Ulterior, o umplem cu literele care nu au fost deja introduse în matrice, în ordine alfabetică. Pentru a se pastra matricea de 5x5 ne trebuie 25 de litere, alfabetul englez are 26 de caractere așa ca înlocuim j-ul cu i.

Cum în RISC-V nu există tablouri bidimensionale, o să folosim un vector de 25 de elemente. Fiind o matrice 5x5 linia se schimbă din 5 în 5 „căsuțe ale vectorului”.

În Ripes nu este implementat space, așa că de preferat să se folosească alt complator (în suportul de laborator space are aceeași semnificație ca pe x86). În acest exemplu cheia este „**SECRET**”, iar mesajul este „**COOPTAT**”.

Deci la final matricea construită arată așa:

```
S E C R T
A B D F G
H I K L M
N O P Q U
V W X Y Z
```

Pentru rezolvarea problemei am inițializat următoarele variabile:

Listing 1: Inițializarea datelor

```
1 .data
2     cheie: .asciiz "SECRET"
3     mesajul: .asciiz "COOPTAT"
4     matrice: .space 25          # matricea 5x5 (liniara)
5     freqv: .space 26           # vector de frecventa
6     encriptat: .space 167      # vectorul cu mesajul codificat
7     decriptat: .space 167      # vectorul cu mesajul decodificat
```

```

8      parcurs: .space 167          # vectorul in care o sa punem in perechi
      mesajul si adaugam X-urile
9      newline: .asciiz "\n"
10
11 .text
12 .global main

```

După ce vom termina de pus literele din cheie in matrice va trebui să o completăm și să ne asigurăm că nu punem aceeași literă de două ori. Pentru a face asta am luat un vector de frecvență cu 26 de spații pentru alfabet. Vectorul v-a fi inițializat cu 0 și pe măsură ce folosim litere îl completăm cu 1. În funcția main inițializăm vectorul:

Listing 2: Initializare vectorul de frecvență

```

1 main:
2     la t0, frecv      # t0 -> adresa frecv
3     li t1, 26         # t1 = 26 litere
4     li t2, 0          # t2 = 0 (contor)
5
6 init_frecv:
7     sb zero, 0(t0)    # frecv[i] = 0
8     addi t0, t0, 1
9     addi t2, t2, 1
10    blt t2, t1, init_frecv #daca contorul < t1 (=26 numarul total de litere
        din alfabet) reia bucla
11                                #altfel trecem mai departe

```

Următorul pas este parcurgerea cheii. Mergem la adresa pointerului cheie. Luăm prima literă din cheie și verificăm dacă e J și dacă e o modificăm cu I, dacă nu face salt direct.

Listing 3: Parcurgere cheie

```

1
2     la s0, cheie      # s0 -> pointer adresa cheie
3     la s1, matrice    # s1 -> adresa baza matrice
4     li s2, 0          # s2 = 0 (pozitie curenta)
5
6 cheie_loop:
7     lb t0, 0(s0)      # t0 = litera curenta din CHEIE
8     beqz t0, start_alfabet # daca '\0', trecem la completarea matricei cu
        literele nefolosite din alfabet
9
10    # daca litera == 'J', o inlocuim cu 'I'
11    li t1, 'J'         # t1 = J
12    li t2, 'I'         # t2 = I
13    beq t0, t1, inlocuire # t0(litera curenta) == t1(j) facem salt
14    j verific_frecv
15
16 inlocuire:
17     mv t0, t2         # t2 era I asa ca t0 devine 'I'
18
19 verific_frecv:
20
21     addi t3, t0, -'A'  # t3(index) = litera curenta - 'A' calculez indexul
        unei litere in alfabet
22     la t4, frecv      # t4 -> pointer adresa vector de frecventa
23     add t4, t4, t3     # pun in vectorul de frecventa l
24     lb t5, 0(t4)      # dau load in t5 la primul element din frecv care e 0
25     bnez t5, skip_key  # daca nu e 0 inseamna ca e folosit si sarim
26
27     add t6, s1, s2     # matrice[pos] = litera
28     sb t0, 0(t6)      # dau store la t5 in adresa primul element din frecv
29

```

```

30     li t5, 1      # freqv[index] = 1
31     sb t5, 0(t4) # dau store la t5 in adresa primul element din freqv
32
33     addi s2, s2, 1    # pos++
34
35 skip_key:
36     addi s0, s0, 1    # s0++ mergem la urmatoarea casuta
37     j cheie_loop      # facem salt si trecem la urmatoare litera din cheie

```

După ce am introdus cheia trebuie sa terminam de completat matricea. Pentru asta incepem cu litera A si verificăm pe rand frecvența până la ultima cifră din alfabet. Folosim aceeasi idee ca la cheie.

Listing 4: Completare cu alfabetul

```

1 start_alfabet:
2     li t0, 'A'      # incarca prima litera din alfabet 'A' in t0 pentru a
                       # incepe verificarea si completarea
3
4 copletare_alfabet:
5     li t1, 'Z'      # t1 = Z pentru a putea forma conditie de iesit
6     bgt t0, t1, et_exit # daca am trecut de Z, gata
7
8     li t2, 'J'      # punem in t2 j ca sa il putem compara
9     beq t0, t2, trecem_pestes # daca t0(litera curenta) este egal cu t2, sarim
                       # peste J
10
11     addi t3, t0, -'A' # calculez indexul unei litere in alfabet
12     la t4, freqv      # t4 -> pointer adresa vector de frecventa
13     add t4, t4, t3     # pun in vectorul de frecventa l
14     lb t5, 0(t4)      # dau load in t5 la primul element din freqv care e
15     bnez t5, trecem_pestes # daca nu e 0 inseamna ca e folosit si sarim
16
17     add t6, s1, s2     # matrix[pos] = litera
18     sb t0, 0(t6)      # dau store la t0(litera curenta) in matrix[pos]
19
20     li t5, 1          # freqv[index] = 1
21     sb t5, 0(t4)      # dau store la t5 in adresa primul element din freqv
22
23     addi s2, s2, 1    # pos++
24
25 trecem_pestes:
26     addi t0, t0, 1    # t0++
27     j copletare_alfabet # facem salt si trecem la urmatoare litera din
                       # alfabet

```

2.2 Procesare mesaj

Următoarea parte din problemă este să pregătim mesajul de codare. Pentru asta trebuie sa eliminăm spațiile și să punem literele mesajului în perechi de câte 2. Dacă avem un număr impar de litere trebuie să-i completăm ultima pereche cu X. Iar, dacă două litere alăturate sunt identice trebuie să inserăm un X între ele.

În cazul nostru **COOPTAT** il despartim în câte două litere: CO OP TA T și vedem că ultima cifră nu are pereche și punem X. Mesajul procesat(pe care il creem cu ajutorul unui vector procesat) devine: procesat - „CO OP TA TX”

Mai există și cazul în care trebuie să inserăm între litere X dacă se regăsește aceeași literă într-o pereche de două ori: **ACEEASI** devine procesat - „AC EX EA SI”

Parcursul mesajului respecta aceeași schema ca până acum doar ca vom „citi” cate 2 litere una dupa alta manual.

Listing 5: Initializare vectorul de frecvență

```

1 preprocess_mesaj:
2     la t0, mesaj          # t0 -> pointer mesaj original
3     la t1, procesat       # t1 -> pointer mesaj procesat aici il cream
4
5 citire_primul_caracter:
6     lb t2, 0(t0)          # t2 = caracter curent
7     beqz t2, et_exit      # daca e '\0', iesim
8
9     li t3, ' '            # daca e spatiu, il ignoram
10    beq t2, t3, skip_primul
11
12    li t3, 'J'             # folosim aceeasi idee ca la cheie pt
13    inlocuire J
14    li t4, 'I'
15    beq t2, t3, inlocuire_j_mesaj1 # daca e J, il inlocuim cu I
16    j primul_ok
17
18 inlocuire_j_mesaj1:
19     mv t2, t4             # t2 = 'I'
20
21 primul_ok:
22     addi t0, t0, 1        # avansam dupa prima litera
23
24 citire_al_doilea_caracter: # identic cu citirea primului din pereche
25     lb t5, 0(t0)          # t5 = a doua litera
26     beqz t5, ultima_litera_singura # daca s-a terminat sirul
27
28     li t3, ' '            # ignoram spatiile
29     beq t5, t3, skip_al_doilea
30
31     li t3, 'J'            # J -> I
32     beq t5, t3, inlocuire_j_mesaj2
33     j comparare
34
35 inlocuire_j_mesaj2:
36     mv t5, t4
37
38 comparare:
39     beq t2, t5, inserare_X # daca literele sunt egale inseram X intre ele
40
41     sb t2, 0(t1)          # altfel le scriem pe amandoua
42     sb t5, 1(t1)
43     addi t1, t1, 2
44     addi t0, t0, 1        # consumam a doua litera
45     j citire_primul_caracter
46
47 inserare_X:                # cazul 2 din explicatie cand aveam aceeasi lit
48     in pereche
49     # scriem litera + X
50     sb t2, 0(t1)
51     li t6, 'X'
52     sb t6, 1(t1)
53     addi t1, t1, 2
54     # NU consumam a doua litera
55     j citire_primul_caracter
56
57 skip_primul:
58     addi t0, t0, 1
59     j citire_primul_caracter
60
61 skip_al_doilea:
62     addi t0, t0, 1        # incrementam cu 1

```

```

61      j citire_al_doilea_caracter      # trecem sa citim al doilea caracter din
        pereche
62
63 ultima_litera_singura: # primul caz cand mesajul are un numar impar de litere
64
65      sb t2, 0(t1)          # a ramas o singura litera -> adaugam X
66      li t6, 'X'
67      sb t6, 1(t1)
68      addi t1, t1, 2
69      j finish
70
71 finish:
72      sb zero, 0(t1)        # terminator de sir
73      ret

```

2.3 Codarea perechilor

Următorul pas e să luăm noua variantă a mesajului nostru, adică vectorul **procesat** și pe rând codăm fiecare pereche din el. Pentru asta ne trebuie matricea pe care am construit-o. în cazul nostru:

```

S E C R T
A B D F G
H I K L M
N O P Q U
V W X Y Z

```

Pentru a coda mesajul avem 3 reguli de urmat:

1. Dacă literele se afla pe aceeași linie În matrice, atunci fiecare dintre ele va fi Înlocuita cu litera din dreapta din matrice (de pe aceeași linie)
2. Dacă cele doua litere se afla pe aceeași coloana, se va înlocui fiecare cu litera de sub.
3. Dacă literele nu se afla nici pe aceeași linie, nici pe aceeași coloana, atunci ele sunt colturile unui dreptunghi. Vor fi înlocuite cu celelalte doua colturi ale dreptunghiului

Mesajul nostru procesat este „**CO OP TA TX**” așa ca îl vom coda astfel:

- Prima preche: CO - în matricea noastră C și O nu se regăsec nici pe aceeași lini, nici pe aceeași coloană așa că vom folosi regula 3. Dreptunghiul care le conține pe amândouă ca colț este ECPO deci **CO** devine **EP**
- A doua pereche: OP - în matrice sunt pe aceeași linie una lângă alta deci folosim regula 1. O se înlocuiește cu litera din dreapta și va fi P, la fel și P devine Q, deci **OP** devine **PQ**
- A treia pereche TA- în matricea noastră T și A nu se regăsec nici pe aceeași lini, nici pe aceeași coloană așa că vom folosi regula 3. Dreptunghiul care le conține pe amândouă ca colț este STAG deci **TA** devine **SG**
- ULtima pereche: TX-în matricea noastră T și X nu se regăsec nici pe aceeași lini, nici pe aceeași coloană așa că vom folosi regula 3. Dreptunghiul care le conține pe amândouă ca colț este CTZX deci **TX** devine **CZ**

Mesajul nostru după codare este „**EP PQ SG CZ**”.

Pentru codare și decodare trebuie să lucrăm pe o matrice, dar am folosit un vector așa că pentru a parcurge pe „linii” și „coloane” vom folosi formula: **index = lin * 5 + col**

Listing 6: Implementarea formulei în RISC-V

```

1  li t0,5
2  mul t1, lin, t0
3  add t1, t1, col

```

O mică schemă nefinalizată a codului:

Listing 7: Schemă codare perechi

```
1 encriptare_mesaj:
2     la t0, procesat      # t0 = pointer adresa mesaj procesat
3     la t1, encriptat     # t1 = pointer adresa mesaj criptat
4
5 encriptare_loop:
6     lb a0, 0(t0)         # a0 = prima litera
7     beqz a0, incheiere_enc # daca a0 == '\0', iesim din bucla
8     lb a1, 1(t0)         # a1 = a doua litera
9
10    # apel functie care cripteaza o pereche
11    call encriptare_perechi # returneaza a2, a3
12
13    sb a2, 0(t1)          # scriem prima litera criptata
14    sb a3, 1(t1)          # scriem a doua litera criptata
15
16    addi t0, t0, 2        # urmatoarea pereche
17    addi t1, t1, 2        # salt la inceputul loop-ului
18    j encriptare_loop
19
20 incheiere_enc:
21     sb zero, 0(t1)       # am terminat ecriptarea si punem terminator de sir
22
23 encriptare_perechi:
24     # a0 = litera 1 din pereche
25     # a1 = litera 2 din pereche
26
27     # cautam pozitia primei litere
28     call find_position   # obtinem lin1, col1
29
30     # cautam pozitia celei de-a doua litere
31     call find_position   # obtinem lin2, col2
32
33     # daca sunt pe aceeasi linie mutam la dreapta
34     # daca sunt pe aceeasi coloana mutam in jos
35     # altfel: dreptunghi (schimb coloanele)
36     # citim noile litere din matrice
37     # le punem in a2 si a3
38     ret
39
40 #find_position:
41
42     # parcurgem toata matricea intr-un loop
43     # folosim beq pentru a cauta litera pe care o cautam
44     # daca e aia care trebuie luam indexul si cu ajutorul formulei de mai sus
45     # calculam lin si col si iesim din bucla (index = lin * 5 + col)
46
47     #daca nu e trecem la urmatoarea litera pana terminam alfabetul
48     # conditie de intoarcere la inceputulo buclei de find_position
```

2.4 Decodarea

Decodarea se face respectând regulile pe care le-am scris mai sus doar că în sens invers. Primim un mesaj pe care îl împărțim în perechi de câte două litere și cu ajutorul matricei construite ne uităm la poziția lor în matrice și folosim următoarele reguli:

1. Dacă cele două litere se află pe aceeași linie, fiecare literă este înlocuită cu litera din stânga ei.

2. Dacă cele două litere se află pe aceeași coloană, fiecare literă este înlocuită cu litera de deasupra ei.
3. Dacă literele nu sunt nici pe aceeași linie, nici pe aceeași coloană, ele formează colțurile unui dreptunghi.

După procesarea tuturor perechilor, se obține mesajul decodat. Eventualele caractere X inserate la criptare pot fi eliminate în funcție de context.

2.5 Final

Proiectul a avut ca scop implementarea cifrului Playfair utilizând arhitectura RISC-V. Chiar dacă implementarea nu este finalizată în totalitate am reușit să aprofundez limbajul RISC-V prin utilizarea buclor, salturilor condiționate, lucrul cu registrele și apelurile de funcții.

De asemenea, proiectul a reprezentat o bună oportunitate pentru a înțelege un mecanism nou de criptare, respectiv cifrul Playfair, și regulile sale de funcționare.